

Operating Systems (CSE2430)

Lab 6: File systems

March 13, 2025

Deadline: 28 March 2025 - 12:30

Introduction

Lab 6 will focus on the file system, through the introduction of *File Systems in User Space* (FUSE). You will have access to a data structure called the DFS, and will have to make it accessible through the file system. In order, the purposes of this lab session are for you to:

- Become familiar with FUSE.
- Become familiar with basic file and directory operations.
- Make an underlying data structure accessible through FUSE.

Before you start working on the assignments, please **very carefully** read the provided explanations, **and familiarize yourself with the provided code**, as this will save you immeasurable time.

File Systems in User Space (FUSE)

FUSE, short for File Systems in User Space ¹, is an interface for programs in user space to create and present virtual file systems. Traditionally, file systems are built within kernel space, which (as you may remember from a previous assignment) brings with it several limitations. As such, FUSE comes with a number of advantages and disadvantages:

- A FUSE file system can use userspace code and libraries.
- A FUSE file system can be loaded, mounted and unmounted by ordinary users.
- If a FUSE file system crashes, it will not crash the kernel.
- A FUSE file system is likely far slower than a kernel-based file system.

FUSE file systems are often used for writing virtual file systems, which can act as a specific view or translation of an existing underlying file system or other such component. For example, two file systems built with FUSE are *GmailFS* ² (which stores data in e-mail format on Gmail servers) and *SSHFS* ³ (which provides access to data on a remote server through *SSH*).

A file system built with FUSE can be mounted to a directory by executing its built program. For example:

```
$ ls
    dir fuse_program.o fuse_program.c
$ ./fuse_program dir
```

Afterwards, the file system is directly accessible in the folder into which it is mounted:

```
$ cd dir/
$ ls
    subdir1 subdir2 Hello-there.txt
```

To unmount the file system again, `fusermount` or `umount` can be used:

```
$ cd ..
$ sudo umount dir/
```

¹FUSE: <https://github.com/libfuse/libfuse>

²GmailFS: <https://sr71.net/projects/gmailfs/>

³SSHFS: <https://github.com/libfuse/sshfs>

FUSE File operations

To function as a FUSE file system, a program must implement a number of *operations* that mirror common file system operations. FUSE has both a high-level and a low-level API for this. In this lab-session, we will use the high-level API. There is documentation⁴ available online for specifics. Some of the high-level operations are shown below:

```
struct fuse_operations {
    int (*getattr) (const char *, struct stat *);
    int (*mkdir) (const char *, mode_t);
    int (*unlink) (const char *);
    int (*rmdir) (const char *);
    int (*create) (const char *, mode_t,
        struct fuse_file_info *);
    int (*rename) (const char *, const char *);
    int (*truncate) (const char *, off_t);
    int (*read) (const char *, char *, size_t, off_t,
        struct fuse_file_info *);
    int (*write) (const char *, const char *, size_t, off_t,
        struct fuse_file_info *);
    int (*link) (const char *, const char *);
};
```

For a specification of all the operations FUSE supports, please refer to the specific documentation⁵. As stated previously, almost all of these operations map directly to existing operations in the C library. For example: `mkdir()` creates a directory, while `rename()` renames a file. Some operations are different, however, such as `getattr()`, which is similar to the existing `stat()` in C.

For the coming assignments, it is important that you understand the basics behind the following FUSE operations, as you will have to implement parts of them. Any function arguments marked with `...` are not relevant for this lab session.

- `int os_mkdir(const char *path, ...)`
This function shall create a new directory with the name `path`.
- `int os_rmdir(const char *path)`
This function shall remove an empty directory whose name is given by `path`.
- `int os_create(const char *path, ...)`
This function shall create a file with the name `path`.
- `int os_unlink(const char *path)`
This function shall remove a file at `path`.
- `int os_read(const char *path, char *buff, size_t size, off_t off, ...)`
This function shall read data from a file at `path`. Specifically, it reads a chunk of bytes of size `size` from the file into `buff`, starting from offset `off`. It returns the exact number of

⁴FUSE Documentation: <https://libfuse.github.io/doxygen/>

⁵`fuse_operations`: https://libfuse.github.io/doxygen/structfuse__operations.html

bytes that was read (this is not always the same as `size`, as the function can encounter an end of file).

- `int os_write(const char *path, char *buff, size_t size, off_t off, ...)`

This function shall write data to a file at `path`. Specifically, it writes a chunk of bytes of size `size` from `buff` to the file, starting from offset `off`. The file is grown as needed, but never shrunk. It returns the exact number of bytes that was written.

- `int os_readdir(const char *path, void *buff, fuse_fill_dir_t fill, ...)`

This function provides directory data for a directory at `path`. Specifically, it fills `buff` with the correct number of items in the directory (subdirectories, files) through use of `fill`, which is a function.

For example, to add a subdirectory *documents* to `buff`, use `fill(buff, "Documents", NULL, 0);`.

The DFS

For these assignments, you will be implementing the mentioned FUSE operations to work on the DFS. The DFS is a tiny and completely in-memory data structure that stores directories and files in a tree-like structure. You can read the code, which is located in `dfs.c`, but you do not have to ever modify it for the assignment. The basic structures of the DFS are `DfsDir` and `DfsFile`.

Directories store entries, which can be subdirectories and files. Files store their contents as a length and an array of chars, not necessarily a string.

Furthermore, the DFS has a number of functions that can be used to, among others, add or remove files and directories, or search through the structure. The functions listed below should be sufficient for navigating within DFS, but the actual code and header files (`dfs.h`, `dfs-helpers.h` and `dfs-path.h`) contain much more detail.

```
// Parses a string path to a DFS path. You will need DfsPath for navigation.
DfsPath *dfs_parse_path(const char *str) ATTRS(nonnull);
```

```
// Takes a DFS path, removes the last component of the path and optionally
// returns the removed component, aka the name.
// This can be used to get the parent path and a file/directory name.
DfsStatus dfs_path_pop(DfsPath *path, char **name_p);
```

```
// Looks up a directory by following the given path from root_dir.
DfsStatus dfs_find_dir(DfsDir *root_dir, DfsPath *path, DfsDir **dir_p)
```

```
// Looks up a file by following the given path from root_dir.
DfsStatus dfs_find_file(DfsDir *root_dir, DfsPath *path, DfsFile **file_p);
```

```
// Creates an empty file. Returns pointer to file.
DfsFile *dfs_create_file(void);
```

```
// Creates a directory. Returns pointer to directory.
DfsDir *dfs_create_dir(void);
```

```
// Adds a file to a given directory.
DfsStatus dfs_add_file(DfsDir *dir, const char *name, DfsFile *file);

// Adds a directory to another given directory.
DfsStatus dfs_add_dir(DfsDir *dir, const char *name, DfsDir *child_dir);

// Removes a file from a directory.
// If file_p is NULL, the file will be destroyed.
DfsStatus dfs_remove_file(DfsDir *dir, const char *name, DfsFile **file_p);

// Removes a child directory from a directory.
// If child_dir_p is NULL, the child directory will be destroyed recursively.
DfsStatus dfs_remove_dir(DfsDir *dir, const char *name, DfsDir **child_dir_p)

// Gets number of entries inside given directory.
size_t dfs_get_dir_size(DfsDir *dir);
```

For (almost) every utility function there is a version that uses a `DfsPath` and a version that accepts a string. You are free to use both, but we recommend you be consistent and use one type only. The path version is more reliable, but requires you to parse the string before using `dfs_parse_path`. The string version is more convenient, but we have seen in past years multiple submissions failing because of it being misused.

The DFS uses a hash map to store directory entries. You can't see the implementation just in case, but a header with documentation is available if you want to interface with it directly.

It should be clear to you that the DFS is extremely limited. It is not a file system, as it has no systems in place for symbolic links, extended attributes, etc. Furthermore, it doesn't persist data, as it is completely in-memory. Finally, it is not even accessible to us from the shell!

This is where FUSE comes in, as we can expose the DFS to a user as if it were a file system.

Debugging your FUSE program

While working on your assignments, you are bound to break something, or encounter a crash. You may wish to add debug statements to see what's happening. However, FUSE has the nasty tendency to not output `printf()` statements while it is used normally. Thankfully, there are workarounds available:

- FUSE programs can be run in the foreground by passing the `-f` or `-d` (which also enables FUSE debugging) flags, although you will have to find another way to use your file system while it is doing so (if you have multiple shells available through SSH, this is easy).
- `syslog()` can be used to, well, log. For more information on system logging, please see [this link](#) ⁶.
- FUSE programs can be run in `gdb`, which is extremely helpful when debugging crashes. For example, to get started, you can use:

⁶Linux Log File Locations: <https://www.cyberciti.biz/faq/linux-log-files-location-and-how-do-i-view-log>

```
$ gdb os_assignment
...
(gdb) r -d dir/
...
```

Assignments

For the assignments, you will use the files found on Brightspace. You can copy these files to your development environment (docker, own system, VM, ...). You might need to install some dependencies, mainly `xxhash` and `fuse` development packages. First create a new directory for this assignment and change to it with `cd`. Then copy all the files into your directory.

You need to build the hashmap library files, this is achieved by doing:

```
$ make -f hashtable.Makefile
```

This needs to be done only once, unless `make clean` is run.

For the assignments, you should modify `os_assignment.c`, which you have to hand in on Weblab. A new **makefile** is available in the same directory. **To compile your code, run `make os_assignment`.**

Do not use any global variables aside from the pre-defined `root_dir`. We call the `create_dfs` function to reset your filesystem multiple times while testing, so any additional state may cause your submission to fail.

As you can see in `os_assignment.c`, a part of the FUSE code (for example, `truncate()` and `getattr()`) is already in place, but many essential operations are not implemented. During these assignments, you will implement these specific functions we require to interact with DFS through the normal file system. Until you do so, large parts of your file system will not work, or will be invisible to you!

Please read the assignments carefully and make sure your code exactly implements the given descriptions. We will assess the functionality and clarity of your code and comments.

Please read **FUSE File operations** and **The DFS** (and probably the attached code) before starting on the assignments. You can try to complete the assignments very quickly and with very short code, without ever modifying strings or traversing any data structure manually, by using the functions from the DFS. Doing this can save you immeasurable time.

6.1: Reading directories

The first function you will have to complete is the function `os_readdir()`. As explained previously, you can use `fill()`, which is provided by FUSE, to add the correct items to `buff`. Some (incorrect) code is already inside this function, to show you some simple usage of DFS.

Your implementation should:

1. Fill the buffer with `”.”`, which is a hard link from a folder to itself, and `”..”`, which is a hard link from a folder to its parent.
2. Fill the buffer with all subdirectories and files in the directory specified by `path`.

If your implementation is correct, you can now see all the files and folders defined in `create_dfs()`, through `ls` in the shell. Beforehand, you could see a fake folder called `”Documents”`.

6.2: Adding and removing directories

Now, you will have to implement `os_mkdir()` and `os_rmdir()`.

Your implementation should:

1. In `os_mkdir()`, find the parent directory to add your new directory to, create the new directory, and add it to the parent directory.
2. In `os_rmdir()`, find a directory and remove it, but only if it is empty.

If your implementation is correct, you can now add and remove directories to and from your file system, through the shell.

6.3: Reading and writing files

Now, you will have to implement `os_read()` and `os_write()`. Usage of `memcpy()` and `realloc()` is recommended!

Your implementation should:

1. In `os_read()`, find the file to read from, copy the chunk of bytes to the buffer, and return the correct number of read bytes. If the file does not exist, do not forget to return an error.
2. In `os_write()`, find the file to write to, copy the chunk of bytes from the buffer into the file, and return the correct number of written bytes. If the file does not exist, do not forget to return an error. If the file is too short, make it larger. But, a file can not shrink by writing to it!

If your implementation is correct, you can now open up one of the arbitrary files in your system (say, in `nano`), add some text to it and then save it.

6.4: Adding and removing files

Now, you will have to implement `os_create()` and `os_unlink()`.

Your assignment is to:

1. In `os_create()`, find the parent directory where you create a file, create the new file, and add it to the directory.
2. In `os_unlink()`, find the file and remove it.

If your implementation is correct, you can now add and remove simple files to your system. For example:

```
$ echo "Hello!" > hi.txt
$ ls
      subdir readme.txt hi.txt
$ rm hi.txt
$ ls
      subdir readme.txt
```

Bonus: hard links

Now that your filesystem can store trees of arbitrary data, it's time to take the next step: links. Links allow a filesystem to have different paths to the same file or directory. This saves space, simplifies paths and allows for easier system maintenance.

In this bonus, you'll be implementing what's known as *hard links*.

Hard links, created with the `link` syscall, create an additional direct reference to an existing file. Effectively, the file will have more than one parent directory. In DFS, creating a hard link means adding an existing `DfsFile` instance to another directory without removing it from the previous one. In order to make sure a linked file remains valid even if one entry is removed, you'll need to count the references to the file. This count must also be returned by `os_getattr` as the `st_nlink` field. You do not have to support anything other than hard links to files.

Your FUSE code needs to correctly handle symbolic links in the existing functions `os_unlink` (which needs to decrease the reference count and only delete the file if this reaches zero) and `os_getattr` (which needs to correctly set `st_nlink`), as well as provide an implementation for `os_link`.

You probably will have to modify the DFS source code in order to add this. When submitting, include these files as well.

Your objective is to:

1. Modify DFS to include reference counting for files. (You can ignore thread safety)
2. Enhance `os_unlink` and `os_getattr` with link support.
3. Implement the `link` FUSE operation using these DFS enhancements.